Project Documentation

Project Title: Low-Cost Radiology Using AI Techniques
Date: 3/10/2026

Project Workflow

Data Collection → Data Preprocessing → Model Selection → Model Training

→ Model Evaluation → Performance Metrics → Deployment → Backend → Frontend
Feasible study:

Feasibility Study

The feasibility study for this project includes the following steps:

1. Identify the type of medical imaging data.
2. Study different types of medical imaging datasets.
3. Collect internet sources and dataset links.
4. Compare different deep learning models.
5. Analyse the number of parameters and efficiency according to project needs.
6. Finalise the most suitable model.
7. Train the model and split the dataset into training, validation, and testing sets.
8. Select evaluation metrics.
9. Evaluate the trained model and prepare for deployment.
10. Implement backend and frontend systems.
11. Prepare the project report and document challenges faced.

Type of Data Used

Medical Imaging Data (X-ray Images)

Specifically, chest **X-ray images** because they are:

- Widely available in public datasets
- Common in rural healthcare diagnosis
- Easier to train models on compared to CT/MRI

Each image typically includes a **label** describing the condition.

Example:

Image	Label
X-ray image	Normal
X-ray image	Pneumonia
X-ray image	Tuberculosis

This is called **supervised learning data**.

The dataset will contain:

Images

Formats usually include:

- .jpg
- .png
- .jpeg

Example structure:

```
dataset/
  train/
    normal/
    pneumonia/
  test/
    normal/
    pneumonia/
```

Metadata

Some datasets also contain additional information:

Patient Age	45
Gender	Male
Disease	Pneumonia
Image ID	img_00123

An **X-ray** is a 2D transmission image created by passing high-energy electromagnetic radiation through the body to a detector. It produces a "negative" image based on tissue density: dense structures (bones) appear white, soft tissues appear grey, and air appears black. It is primarily a 2D diagnostic tool used for imaging internal

Grey-scale image

0-255 pixel value

Model selection

Model	Parameters	Input Image Size	Images Needed (Approx.)	Common Usage
LeNet	~60K	32×32	1K – 5K	Digit recognition, simple image classification
AlexNet	~60M	224×224	10K – 50K	Early large-scale image classification
VGG16	~138M	224×224	50K – 200K	Object recognition, feature extraction
ResNet50	~25.6M	224×224	10K – 50K	Medical imaging, object detection
DenseNet121	~8M	224×224	5K – 20K	Medical image analysis, disease detection
InceptionV3	~23.8M	299×299	20K – 80K	Image recognition, computer vision tasks
Xception	~22.9M	299×299	20K – 80K	Advanced image classification
MobileNetV2	~3.5M	224×224	3K – 10K	Mobile apps, edge devices
EfficientNet (B0)	~5.3M	224×224	3K – 15K	Medical imaging, efficient AI systems
Vision Transformer	~86M	224×224	100K+	Large-scale vision tasks

MobileNetV2 was selected because it achieves high accuracy while maintaining a low parameter count, making it suitable for low-cost AI systems deployed in rural healthcare environments.

The **evaluation metric names** you can list in this project:

- Accuracy: the **evaluation metric names** you can list in your project:
- Precision: Precision measures **how many predicted positive cases are actually correct**.
- Recall (Sensitivity): Recall measures **how many actual disease cases are correctly detected**.
- F1 Score: The **F1 Score** balances precision and recall.
- Specificity
- ROC-AUC Score: The **Receiver Operating Characteristic** measures the model's ability to distinguish between classes.
- Confusion Matrix

Number of times we will train the model

train the model only once, but the dataset contains images of **different diseases with labels**.

Example classes:

Normal
Pneumonia
Tuberculosis
COVID-19

During training, the model learns patterns for **all diseases simultaneously**.

So:

Number of training sets = 1.

Advantages:

- Faster development
- Easier deployment in web/mobile apps
- More realistic for hospital use

This is the approach usually used with models.

A single multi-class model will be trained to detect multiple diseases from chest X-ray images, reducing computational cost and simplifying deployment.

Feature	One Model (Multi-Class)	Multiple Models (One per Disease)
Training Process	Train **once** on a dataset containing all diseases	Train **separate models** for each disease
Number of Models	1	Number of diseases (e.g., 3 diseases → 3 models)
Dataset Structure	Images labelled as **Normal, Pneumonia, TB, etc.**	Each dataset contains **only one disease vs normal.**
Training Time	Faster	Much longer
Memory Usage	Low	High
Deployment	Easier for web/mobile apps	More complex
Prediction Process	Model directly predicts disease class	Each model checks the image separately
Maintenance	Easier to update	Harder to maintain
Accuracy	Good for balanced datasets	Sometimes, it is better for rare diseases

Deployment process

First, I trained a deep learning model using a framework like **TensorFlow** or **PyTorch**.

model.h5

Create an API for the Model

You build a backend server that loads the trained model and processes images.

framework:

- **Flask**

The API will:

1. Receive X-ray image
2. Preprocess image
3. Run prediction
4. Return result

Build the Web Interface

Create a simple frontend that allows a doctor to upload an X-ray.

Technologies:

- Python
- HTML
- CSS

Implementation

We will use

MobileNetV2 ~3.5M 224×224 3K – 10K Mobile apps, edge devices

Chosen dataset

https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database

A team of researchers from Qatar University, Doha, Qatar, and the University of Dhaka, Bangladesh, along with collaborators from Pakistan and Malaysia, and physicians, has created a database of chest X-ray images of COVID-19-positive cases, along with Normal and Viral Pneumonia images. The COVID-19, normal and other lung infection datasets are released in stages. In the first release, we have released 219 COVID-19, 1341 normal and 1345 viral pneumonia chest X-ray (CXR) images. In the first update, we have increased the COVID-19 class to 1200 CXR images. In the 2nd update, we have increased the database to 3,616 COVID-19 positive cases, 10,192 Normal, 6,012 Lung Opacity (non-COVID lung infection), and 1,345 Viral Pneumonia images, along with their corresponding lung masks. We will continue to update this database as soon as we receive new X-ray images of COVID-19 pneumonia patients.

Class	Meaning
COVID	COVID-19-infected lungs
Lung Opacity	Lung infection patterns
Normal	Healthy lungs
Viral Pneumonia	Viral lung infection

To keep the model simple, you should select **3 classes**:

Normal
COVID
Viral Pneumonia

Typical split:

70% training
15% validation
15% testing

Image Preprocessing for MobileNetV2

Input size:

224 × 224

Steps:

1. Resize images
2. Normalise pixel values
3. Data augmentation

Examples of augmentation:

- Rotation
- Horizontal flip
- Zoom

After discussing with my mentor, I decided to use the area localisation feature.

So I will use Grad-CAM with CNN.

Transfer learning is a machine learning technique in which a model trained for one task is reused as the starting point for a related second task.

After training the model, I got an accuracy of 92%

I ran about 5 test cases on random X-rays and obtained accurate results.

I generated a heatmap to help the patient interpret the results more effectively with minimal resource usage.

I will upload the performance metrics, but at this moment, I am thinking why the project is limited to only chest x-ray, why not fractures, why not Ultrasound?

After several days of work, today is 16/3/2026.

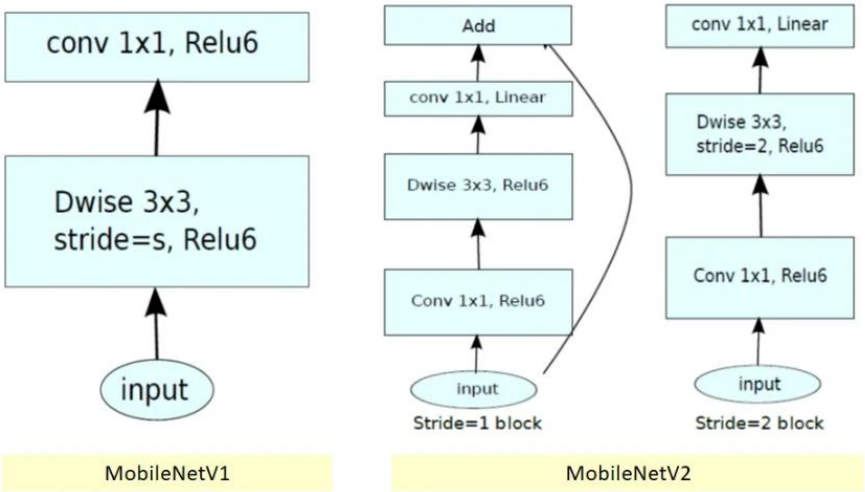
I have successfully implemented fracture and breast ultrasound examination features with the chest X-ray feature.

I obtained the dataset from Kaggle and another site; I mentioned them on GitHub with links.

The accuracies and performance matrices showed good results.

Before adding anything to evaluation and deployment, I want to share some details about the core architecture of MobileNetV2 and the concept of transfer learning.

MobileNetV2 Convolutional Blocks



MobileNet V2 Architecture and Working

MobileNet V2 is a lightweight convolutional neural network designed for **mobile and embedded devices**. It improves efficiency using two key ideas:

- **Inverted Residual Blocks**
- **Linear Bottlenecks**

These techniques reduce computational cost while maintaining high accuracy.

---

1. Overall Architecture of MobileNet V2

MobileNet V2 consists of:

1. Initial **standard convolution layer**
2. Multiple **inverted residual bottleneck blocks**
3. **1×1 convolution layer**
4. **Global average pooling**
5. **Fully connected (classification) layer**

Input Image

↓

3×3 Convolution

↓

Inverted Residual Bottleneck Blocks

↓

1×1 Convolution

↓

Global Average Pooling

↓

Fully Connected Layer

↓

Output Classes

MobileNet V1 vs MobileNet V2 (Short Comparison)

Feature	**MobileNet V1**	**MobileNet V2**
Main Idea	Depthwise Separable Convolution	Inverted Residual + Linear Bottleneck
Architecture	Simpler	More advanced
Residual Connections	Not used	Used (skip connections)
Feature Learning	Basic	Better feature representation
Information Loss	Higher due to ReLU everywhere	Reduced due to a linear bottleneck
Accuracy	Slightly lower	Higher than V1
Efficiency	Lightweight	More efficient and optimised

We used transfer learning by loading a pretrained MobileNetV2 model with ImageNet weights and replacing the top classification layers to train the model on the chest X-ray dataset.

How transfer learning works.

Transfer learning is a technique in which a pre-trained model (trained on a large dataset such as ImageNet) is reused for a new, **related task**. Instead of training a model from scratch, we **transfer learned knowledge to solve a different** problem.

Pre-trained Model

A deep learning model (like **MobileNetV2, ResNet, VGG**) is first trained on a **large dataset** such as ImageNet.

During this training, the model learns:

- edges
- textures
- shapes
- patterns

These are **general image features** that are useful for many tasks.

Feature Extraction

When we apply transfer learning:

- We **keep the early layers of the pre-trained model**
- These layers already know how to extract useful features from images.

Replace Final Layers

The original model was trained for **different classes** (e.g., 1000 ImageNet classes).

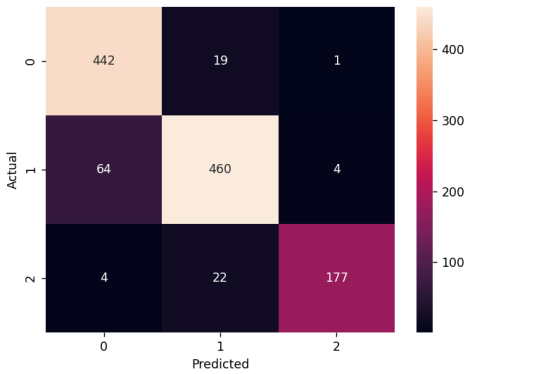
So we:

- **remove the last classification layer**
- **add a new fully connected layer** for our task.

Let's talk about evaluation and results.

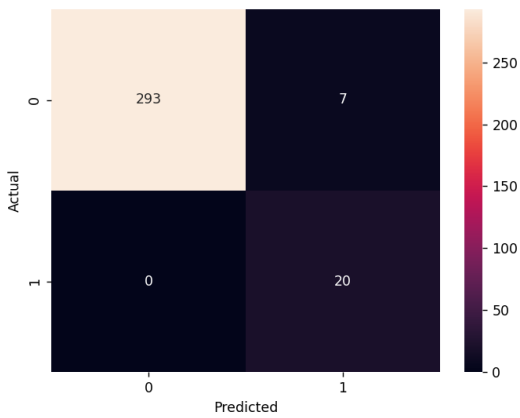
For chest X-ray.

```
38/38 ━━━━━━━━━━━━━━━━━━━━ 27s 674ms/step
Accuracy: 0.9044425817267393

Classification Report:
                precision    recall  f1-score   support

         COVID       0.87      0.96      0.91       462
        Normal       0.92      0.87      0.89       528
Viral Pneumonia      0.97      0.87      0.92       203

      accuracy                           0.90      1193
     macro avg       0.92      0.90      0.91      1193
  weighted avg       0.91      0.90      0.90      1193


Confusion Matrix:
[[442  19   1]
 [ 64 460   4]
 [  4  22 177]]
```



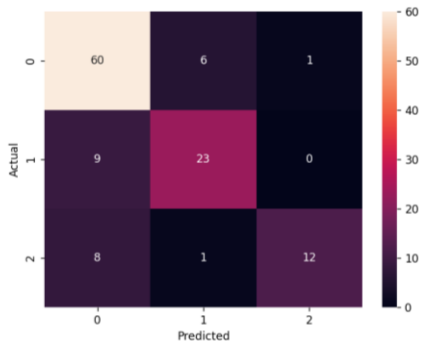
For fracture X-ray.

```
Found 320 images belonging to 2 classes.
10/10 ━━━━━━━━━━━━━━━━━━━━ 8s 723ms/step
Accuracy: 0.978125

Classification Report:
              precision    recall  f1-score   support

    fracture       1.00      0.98      0.99       300
      normal       0.74      1.00      0.85        20

    accuracy                           0.98       320
   macro avg       0.87      0.99      0.92       320
weighted avg       0.98      0.98      0.98       320


Confusion Matrix:
[[293   7]
 [  0  20]]
```



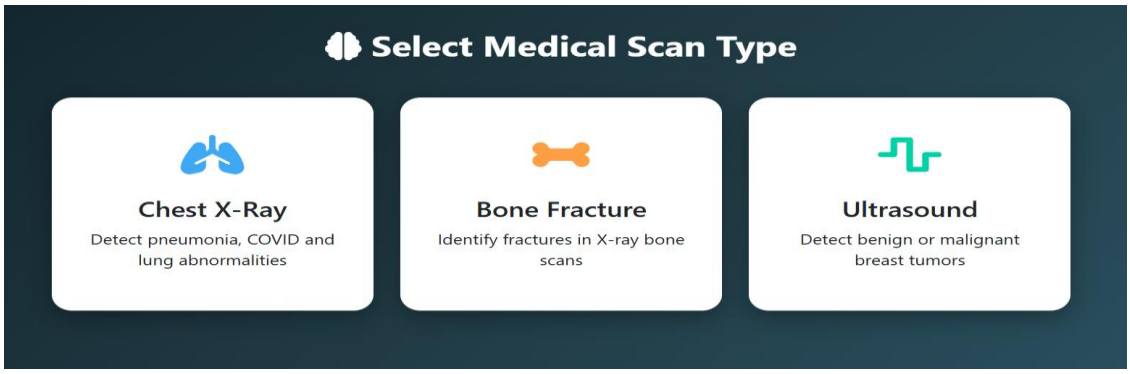
For ultrasound.

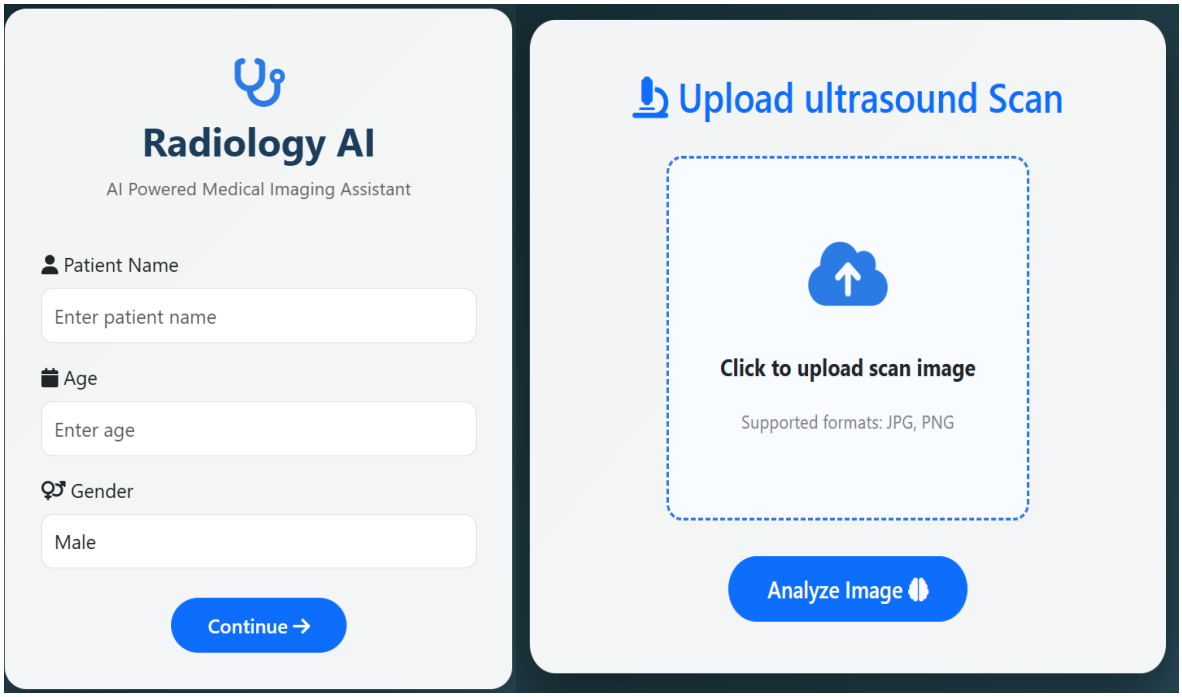
```
Accuracy: 0.7916666666666666

Classification Report:
              precision    recall  f1-score   support

      benign       0.78      0.90      0.83        67
   malignant       0.77      0.72      0.74        32
      normal       0.92      0.57      0.71        21

    accuracy                           0.79       120
   macro avg       0.82      0.73      0.76       120
weighted avg       0.80      0.79      0.79       120


Confusion Matrix:
[[60  6  1]
 [ 9 23  0]
 [ 8  1 12]]
```



After receiving these results, I built a web page using Flask, deployed my trained models, and gave the project a clean, user-friendly look.

I have shared some images to give you an idea of how the project looks.





AI Diagnosis Result

Scan Type

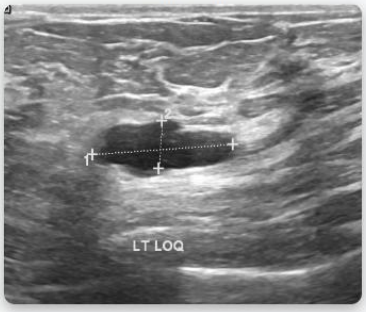
ultrasound

AI Prediction

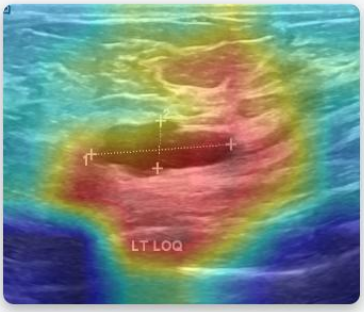
⚠ benign

AI Visualization

Original Scan



AI Heatmap



Heatmap Explanation

- **Red / Pink** – Strong evidence for abnormal region
- **Yellow** – Moderate importance for AI decision
- **Green** – Low importance
- **Blue** – Not important for prediction

AI Suggestion:
This prediction is generated using deep learning analysis. Clinical confirmation by a radiologist is recommended.

← Analyze Another Scan